Current-generation MLFF campaign CLI specification

Version: `mdstats 0.20.242a0` current contract Status: implemented for the P1-P5 functional campaign lifecycle

Purpose and authority

The campaign CLI projects the current source, statistical, target-size, post-selection, and production owners into one restartable user workflow. It does not replace those owners or create a second scientific state machine. Architecture defines ownership and data flow; this specification defines the public command, configuration, persistence, currentness, and failure contract.

The source-checkout entry point is:

```
python tools/mdstats-mlff-campaign.py --config campaign.toml <command>
```

Public command surface

The parser exposes exactly these commands:

```
init
doctor
prepare
select-target-size
cross-validate
train-production
status
advance
guide
storage
```

The scientific lifecycle is:

```
init -> doctor -> prepare -> select-target-size -> cross-validate -> train-production
```

`storage` is orthogonal artifact management. `status` and `advance` derive their projection from the same current owners. The current campaign has no separate pre-screen gate or downstream physical-test command; downstream qualification is a later product boundary and is not dispatched by P6.

No command may silently skip a failed, stale, waiting, or incompatible current authority. A terminal scientific target-size failure is reported as a result and exposes no production next action.

User-visible layout

The operator supplies `campaign.toml`. The workspace contains:

```
<workspace>/campaign-manifest.json
<workspace>/.mdstats/campaign.sqlite3
<workspace>/.mdstats/                # current records and reconstructible caches
<workspace>/data/                     # current MACE materializations
<workspace>/runs/                     # authorized checkpoints and logs
<workspace>/models/                   # current production publication
<workspace>/results/                  # bounded summaries and cleanup reports
```

SQLite rows and content-addressed files are authoritative only through their own current owners. File existence, a caller-held object, or a cache path is not evidence of currentness.

Configuration contract

`campaign.toml` owns requested policy and source locations. Runtime observations and durable scientific outcomes are persisted separately. Paths are resolved relative to the configuration file, and a non-positive execution timeout means no campaign wall-clock timeout.

The generated current configuration exposes:

```toml
[target_data.size_convergence]
target_size_power_min = 7
target_size_power_max = 14
evaluation_size_powers = [8, 9, 10]
fidelity_epochs = [1, 3, 10]

[post_selection.cv]
fold_count = 2
partition_seed = 7
seeds = [11]
max_num_epochs = 2
acceptance_maximum = 0.5

[post_selection.production]
seeds = [5]
```

The configured power ceiling is not a fixed scientific constant. Candidates are additionally bounded by the available `P_train` population and the current policy. Optimizer seeds are authored by the sole enabled training method; there is no separate target-size seed namespace. Pre-target fold-count and partition-seed fields are not current target-size/CV authoring controls.

The learned model precision is `single` (FP32) or `double` (FP64). Critical mdstats reductions, geometry/statistical arithmetic, and persistent bookkeeping remain FP64. The acceleration backend, MACE interface, and runtime identity are bound by the doctor/current protocol record.

Command behavior

`init`

`init` writes an annotated configuration with current target-size and post-selection sections. It must not generate retired lifecycle sections or imply an unconfigured target-size ceiling. Existing configuration files are not overwritten without the explicit force behavior defined by the parser.

`doctor`

`doctor` checks source paths, manifest inputs, foundation/replay identity, package/runtime imports, precision wrappers, requested backend, and available resources. A successful result records the exact runtime realization used by later owners. Unsupported hardware or unavailable long-production resources remain an explicit unavailable/deferred condition, never a fabricated pass.

`prepare`

`prepare` authenticates the manifest and constructs/reuses the current neutral substrate:

```
source/frame/label authority
  -> protected statistical relations
  -> one P_train/M3 split and pi_train/pi_eval
  -> one common target-size preparation
```

It selects no target size, trains no candidate, ranks no checkpoint, and publishes no final model. It is restartable and idempotent when all current inputs and identities match, including when the current generation is already terminal: an unchanged terminal `prepare` is a successful no-op that leaves the canonical generation, its terminal evidence, and its derived result view exactly as they were.

`prepare` is the sole command permitted to interpret live inputs, so it also owns detecting that they changed. Before reusing the stored lower-level catalog it compares the approved manifest digest that catalog was built

from and each source's own byte-identity and control signatures against the files on disk. Materially changed valid inputs are routed through the ordinary catalog reconstruction path and committed as a fresh canonical generation; malformed or unapproved changes still fail under their existing owners. No operator flag is required for this. `--approve-manifest` records the reviewed digest; `--refresh-inferences` refreshes proposed source metadata before approval; `--rebuild-catalog` remains an explicit unconditional reconstruction request, not the mechanism by which ordinary input changes are noticed.

Durable publication is create-or-verify. A prepared component, generation manifest, or normalized frame entry whose content identity already exists on disk is authenticated against that identity – for a frame entry, including every array member's hash, shape, and dtype – before this generation may reuse it. Conflicting or corrupt content fails before adoption and is never overwritten, because another adopted generation may still depend on the bytes that are actually there.

Adoption is compare-and-set against the currentness token captured *before* the expensive construction began, and no campaign-wide writer lock is held across that construction. Two concurrent preparations that produce the same prepared identity converge on one generation; a preparation whose snapshot was built against superseded state fails closed rather than advancing the newer generation.

At the destructive generation boundary, obsolete derived target-size records are detected before semantic decoding, quarantined under a namespace no current loader reads, and rejected rather than migrated. Lower-level source or content caches are reusable only after current-owner revalidation.

`select-target-size`

This command is the sole target-size owner. It runs the configurable ladder and direct evaluation populations through the authenticated continuation

```
n1 / M1 -> n2 / M2 -> n3 / M3
```

with paired optimizer seeds from the sole enabled method. Each candidate is an exact prefix of the one `pi_train` order. The reducer publishes either one `N_selected` with exact `T_selected = pi_train[:N_selected]` or a typed scientific failure. Replay and later validation evidence cannot affect the decision.

Candidate learning-rate amplitude and EMA decay are normalized against the configured reference size (`[target_data.size_convergence.optimizer_normalization]`) so a larger candidate does not also receive more optimizer progress; epoch counts, batch size, LR shape, and every other optimizer setting stay fixed across candidates.

The configured ladder ceiling is a practical budget limit. When `Nmax` remains materially superior to every other successful terminal finalist, it is **selected** and the result carries the non-blocking warning code `nonconverged_at_configured_ceiling`; `status` and the derived result view report the warning alongside the frozen size, the campaign lifecycle is `TERMINAL_SELECTED`, and the next admissible command remains `cross-validate`. Inside the practical-equivalence band the smaller finalist is still preferred. Genuinely insufficient comparison remains a typed scientific failure, and no unconfigured intermediate or rescue size is ever synthesized.

`cross-validate`

This command requires a current terminal selection and constructs the configured `K >= 2` post-selection folds inside exactly `T_selected`. It binds protected relations, fold/seed identities, target-only checkpoint choice, and the all-required-fold/all-required-seed acceptance predicate. It cannot alter the selected size or membership. Missing, stale, or failed fold evidence blocks final production while leaving the selected authority unchanged.

`train-production`

This command requires accepted current post-selection method evidence. It starts fresh from the accepted foundation and trains the complete exact `T_selected` under `[training].max_num_epochs` and the production policy.

A screen or CV checkpoint is never a production parent. Publication rechecks currentness at commit time and cannot promote work from a superseded generation.

`status`, `advance`, and `guide`

`status` projects the current owner states and reports the next safe action. `advance` dispatches only the next current lifecycle owner. `guide` prints the same six-command scientific lifecycle and current configuration/restart semantics. Neither command writes a second scientific authority.

Observation is read-only, coherent, and authenticated. One lifecycle answer reads the target-size revision and every post-selection/qualification pointer row it depends on inside a single campaign-store read transaction, so the ancestry it reports is one that existed: pointer publication mutates campaign metadata without moving the target-size revision, and an answer may never combine a pre-publication view of one stage with a post-publication view of another. Each compact record a pointer names is loaded through its accepted read-only typed store and must reproduce the identity the pointer named before any of its fields – a cross-validation acceptance, a release verdict – influences the report. A missing, unparseable, or misidentified record is reported as blocked. Observation creates no evidence root, opens no provider, and reconstructs no upstream authority.

`storage`

Storage reports and manages only campaign-owned artifacts. Read-only inventory is available through `storage report`. Cleanup tiers require their documented dry-run/apply behavior; archive create, integrity check, and restore are reversible and independently content-checked. External inputs, current scientific records, selected checkpoints, restart state, and diagnostic evidence remain protected.

Persistence and restart

`CampaignStore` shall:

1. use one SQLite database for durable campaign state;
2. serialize canonical payloads with content identities under their owners;
3. record operational stage state without treating it as scientific authority;
4. preserve the latest successful evidence before process exit;
5. retain bounded event history;
6. never use the database location itself as scientific identity.

Current preparation and post-selection owners rederive their input identities on every reopen. A matching completed record is reused only after source, policy, parent, payload, and currentness authentication. A missing, corrupt, stale, or incompatible derived cache is rebuilt by its owner; it is not silently accepted or translated.

Each current write uses a generation-neutral prepare/restart identity. A historical generation marker may be inspected only by the reject-only cutover detector and may not authorize semantic reconstruction. The exact accepted current-generation P5A6 workspace must remain reopenable without a pre-load rewrite; P6 compatibility evidence is separate from fresh P6 restart evidence.

Configuration changes have selective invalidation:

```
target/frame/label/order/ladder/fidelity/common-preparation change
    -> new target-size generation and descendants
post-selection CV-only change
    -> CV and final-production descendants only
production-only horizon/runtime change
    -> final-production descendants only
```

Provenance-only presentation changes do not change scientific arithmetic.

Failure contract

The CLI fails closed for unapproved or changed manifests, missing or incompatible source/foundation/replay inputs, invalid labels, unresolved protected relations, unsupported policy generation, stale current pointers, missing required folds/seeds, corrupt checkpoint/companion state, no admissible checkpoint, target-size lineage mismatch, incompatible persisted state, or a currentness race at publication.

Keyboard interruption returns the parser's documented interruption status and states that authenticated records remain resumable. Errors identify the owning command and the safe corrective action; they do not fall back to a different scientific selector or downstream model.

Acceptance boundary

The current CLI contract is established through the real parser/facade, generated-config parsing, current prepare/selection/CV/production owners, CampaignStore close/reopen, currentness reauthentication, storage cleanup owners, and exact-generation reject-before-reuse tests. The mandatory P5A6-to-P6 qualification additionally authenticates the baseline worktree and import roots before producing state, then opens the unchanged workspace in a separate P6 process.

P6 acceptance is current functional/restart/public-surface closure. It is not GPU, long real-data, M-ladder decision-preservation, deployment, physical, calibration, locked-test, or final-release qualification.